

微积分实际工程应用

摘要

现代 3D 视觉技术（如增强现实（AR）和虚拟现实（VR）的飞速发展，其底层驱动力源自严密的数学理论。主要涉及微积分与高等数学知识在三维空间计算中的核心应用。如多元微积分（如偏导数、链式法则、一阶泰勒展开）如何构建雅可比矩阵，从而实现连续空间的可微渲染与非线性状态估计；同时，展示了线性代数与群论概念（如奇异值分解 SVD、李群与李代数、舒尔补消元法）在解决海量参数优化与高维矩阵降维求解中的“工程化魔法”。

1 第一阶段：传感器输入 (Sensor Input) —— 睁开眼睛

这是整个流水线的源头，负责感知现实世界。

- **动作：** 相机以固定的帧率（如 30 FPS）拍摄现实世界，将光信号转化为数字图像序列。
- **数据形态：** 一张张纯粹的 2D 像素矩阵（RGB 图像或灰度图像）。
- **说明：** 如果是双目（Stereo）或深度（RGB-D）相机，此步骤会直接获取初步的深度信息；若是单目相机（Monocular），则需依靠后续算法估算深度（本文以单目为例）。

2 第二阶段：前端视觉里程计 (Visual Odometry) —— 认路与追踪

前端类似人类的“小脑”，负责处理极短时间内的快速反应，计算相邻两帧之间相机发生的相对运动。

2.1 步骤 1：特征提取 (Feature Extraction)

机器无法逐个处理所有像素，必须找出图像中最具辨识度的“特征点”（如桌角、门框）。

- **提取关键点 (Keypoint):** 寻找图像中梯度变化剧烈的位置 (即二维像素坐标 u, v)。
- **计算描述子 (Descriptor):** 考察关键点周围的像素块特征, 生成一串数字指纹 (如 ORB 的 256 位二进制串), 为后续匹配提供依据。

2.2 步骤 2: 特征匹配 (Feature Matching)

当相机移动拍下第 2 帧图像时, 机器在两帧的特征点之间进行匹配 (对比描述子的汉明距离)。

- **剔除杂音:** 现实中常存在相似纹理干扰导致误匹配。系统采用 RANSAC (随机采样一致性) 算法, 剔除不符合物理运动规律的错配点, 留下高质量匹配对。

2.3 步骤 3: 位姿估计 (Pose Estimation)

获取匹配的像素点对后, 前端根据已知信息的不同, 采用三套核心算法求解相机的运动:

- **2D-2D (初始化阶段):** 系统刚启动且完全无 3D 深度时。使用对极几何 (本质矩阵 E 或单应矩阵 H), 通过八点法解出相机的旋转 R 和平移 t , 接着用三角测量算出这些点的初始 3D 坐标。
- **3D-2D (正常追踪阶段):** 上一帧已建好部分 3D 路标点, 当前帧仅提取 2D 像素点。系统使用 PnP (Perspective-n-Point) 算法, 直接解出当前相机位姿。
- **3D-3D (深度相机专属):** 两帧均具有 3D 坐标, 使用 ICP (Iterative Closest Point) 算法进行匹配。

【核心数学推导】: 2D-2D 前端初始化过程

2.3.1 像素坐标归一化

设图像上的像素坐标为 $\mathbf{p} = [u, v, 1]^T$, 相机的内参矩阵为 K 。为了消除相机内部物理参数的干扰, 将像素坐标映射到焦距为 1 的归一化相机坐标平面上, 记为 $\mathbf{x}$:

$$\mathbf{x} = K^{-1}\mathbf{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (1)$$

2.3.2 对极约束与八点法 (Epipolar Constraint)

设空间点 P 在两帧图像上的归一化坐标分别为 $\mathbf{x}_1$ 和 $\mathbf{x}_2$ ，满足几何刚体变换 $\mathbf{x}_2 \simeq R\mathbf{x}_1 + t$ 。等式两边同时左乘 $t^\wedge$ （平移向量的反对称矩阵），再左乘 $\mathbf{x}_2^T$ ，得到基础的对极约束方程：

$$\mathbf{x}_2^T t^\wedge R \mathbf{x}_1 = 0 \quad (2)$$

令本质矩阵 (Essential Matrix) 为 $E = t^\wedge R$ 。将 3×3 的矩阵 E 展开为 9 维列向量 $\mathbf{e}$ ，收集至少 8 对匹配点构造超定线性方程组 $A\mathbf{e} = 0$ 。通过对 A 进行 SVD 分解，取最小奇异值对应的右奇异向量即可求出 E 。

2.3.3 从本质矩阵恢复 R 和 t

对求得的 E 再次进行 SVD 分解： $E = U\Sigma V^T$ 。引入绕 Z 轴旋转 $\pm\frac{\pi}{2}$ 的矩阵 W 。平移向量和旋转矩阵可由下式求出（结合深度为正的物理先验排查出唯一真解）：

$$t^\wedge = UW(\text{或 } W^T)\Sigma U^T, \quad R = UW^T(\text{或 } W)V^T \quad (3)$$

2.3.4 直接线性变换三角测量 (DLT Triangulation)

已知相机位姿后，计算特征点的 3D 空间坐标 $\mathbf{X} = [X, Y, Z, 1]^T$ 。设两相机的投影矩阵为 $P_1 = K[I|\mathbf{0}]$ 和 $P_2 = K[R|t]$ 。根据投影模型 $\lambda_1 \mathbf{x}_1 = P_1 \mathbf{X}$ 及 $\lambda_2 \mathbf{x}_2 = P_2 \mathbf{X}$ ，利用向量叉乘性质消去未知的深度因子 λ ：

$$\mathbf{x}_1^\wedge (P_1 \mathbf{X}) = 0, \quad \mathbf{x}_2^\wedge (P_2 \mathbf{X}) = 0 \quad (4)$$

联立得到形如 $H\mathbf{X} = 0$ 的方程组，再次利用 SVD 分解求得初始 3D 坐标 $\mathbf{X}$ 。至此，前端给出了一个含有累积误差的相机轨迹和初步的 3D 点云。

3 第三阶段：后端非线性优化 (Back-end Optimization) —— 精雕细琢

前端只管相邻两帧，目光短浅，误差会随着时间不断累积。后端就像“大脑”，负责全局统筹，消除累积误差。

3.1 步骤 1：构建关键帧 (Keyframes)

如果把每一帧（每秒 30 帧）都送给后端优化，电脑会瞬间卡死。所以系统会挑选具有代表性的帧（比如相机移动了一定距离、或者看到了足够多的新特征点）作为关键帧，把冗余的普通帧丢弃。

3.2 步骤 2：光束法平差 (Bundle Adjustment, BA)

将一段时间内的所有关键帧位姿和 3D 路标点全部扔进一个巨大的优化方程中，经过 BA 优化后，相机轨迹和 3D 地图就像被“梳理”过一样，变得异常平滑且精确。

【核心数学推导】：后端 BA 优化过程

3.2.1 后端状态表达：李群与李代数

相机位姿变换矩阵 $T \in SE(3)$ 不满足加法封闭性。因此引入李代数 $\xi = [\rho, \phi]^T \in \mathfrak{se}(3)$ ，通过指数映射转换 $T = \exp(\xi^\wedge)$ 。为优化求导，采用左扰动模型 (Left Perturbation Model)。给当前位姿 T 施加微小左扰动 $\Delta T = \exp(\delta \xi^\wedge)$ ：

$$T_{new} = \exp(\delta \xi^\wedge) T \quad (5)$$

3.2.2 重投影误差与雅可比矩阵推导

设路标点在世界坐标系下为 P_w ，投影到像素平面的理论观测值为函数 $h(\xi, P_w)$ ，实际特征匹配的像素观测值为 $\mathbf{z}$ 。定义重投影误差 $\mathbf{e}(\xi) = \mathbf{z} - h(\xi, P_w)$ 。利用链式法则求误差 $\mathbf{e}$ 对位姿扰动 $\delta \xi$ 的偏导数（雅可比矩阵 J ），设 $P_c = [X_c, Y_c, Z_c]^T = T P_w$ ：

$$J = \frac{\partial \mathbf{e}}{\partial \delta \xi} = \frac{\partial \mathbf{e}}{\partial P_c} \frac{\partial P_c}{\partial \delta \xi} \quad (6)$$

第一项（误差对相机坐标系点 P_c 的偏导， 2×3 矩阵）：

$$\frac{\partial \mathbf{e}}{\partial P_c} = - \begin{bmatrix} \frac{f_x}{Z_c} & 0 & -\frac{f_x X_c}{Z_c^2} \\ 0 & \frac{f_y}{Z_c} & -\frac{f_y Y_c}{Z_c^2} \end{bmatrix} \quad (7)$$

第二项（一阶泰勒展开下，变换点对扰动 $\delta \xi$ 的偏导， $\mathfrak{se}(3)$ 的形式， 3×6 矩阵）：

$$\frac{\partial P_c}{\partial \delta \xi} = \begin{bmatrix} I_{3 \times 3} & -P_c^\wedge \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & Z_c & -Y_c \\ 0 & 1 & 0 & -Z_c & 0 & X_c \\ 0 & 0 & 1 & Y_c & -X_c & 0 \end{bmatrix} \quad (8)$$

两项相乘，得到最终的 2×6 位姿雅可比矩阵：

$$J = - \begin{bmatrix} \frac{f_x}{Z_c} & 0 & -\frac{f_x X_c}{Z_c^2} & -\frac{f_x X_c Y_c}{Z_c^2} & f_x + \frac{f_x X_c^2}{Z_c^2} & -\frac{f_x Y_c}{Z_c} \\ 0 & \frac{f_y}{Z_c} & -\frac{f_y Y_c}{Z_c^2} & -f_y - \frac{f_y Y_c^2}{Z_c^2} & \frac{f_y X_c Y_c}{Z_c^2} & \frac{f_y X_c}{Z_c} \end{bmatrix} \quad (9)$$

3.2.3 非线性优化求解与舒尔补 (Schur Complement)

最小化误差平方和，构造高斯-牛顿法 (Gauss-Newton) 的增量方程： $H \delta \mathbf{x} = -\mathbf{b}$ 。利用海森矩阵 H 的稀疏箭头结构，将其分块为位姿块 (c) 和路标点块 (p)：

$$\begin{bmatrix} H_{cc} & H_{cp} \\ H_{pc} & H_{pp} \end{bmatrix} \begin{bmatrix} \delta \mathbf{x}_c \\ \delta \mathbf{x}_p \end{bmatrix} = \begin{bmatrix} -\mathbf{b}_c \\ -\mathbf{b}_p \end{bmatrix} \quad (10)$$

通过舒尔补操作，消去对角矩阵 H_{pp} 对应的海量路标点增量，得到仅含位姿增量 $\delta \mathbf{x}_c$ 的方程，大幅降低计算维度，实现实时求解：

$$(H_{cc} - H_{cp}H_{pp}^{-1}H_{pc})\delta \mathbf{x}_c = -\mathbf{b}_c + H_{cp}H_{pp}^{-1}\mathbf{b}_p \quad (11)$$

4 第四阶段：回环检测 (Loop Closure) —— 唤醒记忆

即使有后端优化，只要系统运行时间足够长，误差依然会悄悄累积。回环检测就是 SLAM 的“似曾相识 (Déjà vu)”机制。

- **词袋模型 (Bag of Words, BoW)**：系统会把之前看到过的特征点描述子聚类成一本“字典”。当相机来到一个新地方时，它会查字典，判断“这个地方我以前是不是来过？”
- **全局位姿图优化 (Pose Graph Optimization)**：如果系统确信自己回到了曾经走过的地方（比如绕了一圈回到了原点），它会把当前的轨迹强行“拉”回原点，并将误差均匀地分摊到整个闭环轨迹的每一个节点上，瞬间消除所有累积漂移！

5 第五阶段：建图 (Mapping) —— 描绘世界

最后，机器要把算出来的 3D 路标点呈现出来供人类或机器使用。

- **稀疏地图 (Sparse Map)**：就是前端和后端共同维护的那些零散的 3D 特征点。它看起来像一堆骨架，人类很难看懂这是什么，但对于扫地机器人或无人机用来定位和避障已经足够了。
- **稠密地图 (Dense Map)**：如果你想用在 AR/VR 或者 3D 重建中，稀疏点云是不够的。此时需要开启稠密建图线程，计算图像上每一个像素的深度，拼接成密密麻麻的点云，甚至最终生成带纹理的三维网格 (Mesh) 模型。